Re-Design of GTE in HTML5/JavaScript

The Project

Proposed Project

Re-design of GTE in HTML5/Javascript
Game theory provides mathematical concepts and tools for modeling and analyzing interactive scenarios.The central concept for non-cooperative games is the Nash equilibrium which prescribes a strategy for each player that is optimal when the other players keep their prescribed strategies fixed.

The project would involve a redesign of the JSGTE repository. It aims at providing browser front-end for creating game trees and solving games in strategic form.The game is solved by computing its Nash Equilibria via various algorithms that are implemented in Java on a jetty server. The browser part is currently implemented as a flash program in ActionScript. It has been discontinued as the code became too monolithic. So the current aim is to rewrite the repository in HTML5 and Javascript with better modularisation.

JSGTE can even be used just as a drawing tool for games, which can be exported as pictures to file formats for use in papers or presentations.

The following pages describe some tentative screen layout (wireframe) for the game tree and the strategic form, as currently implemented in the GTE Flash version.

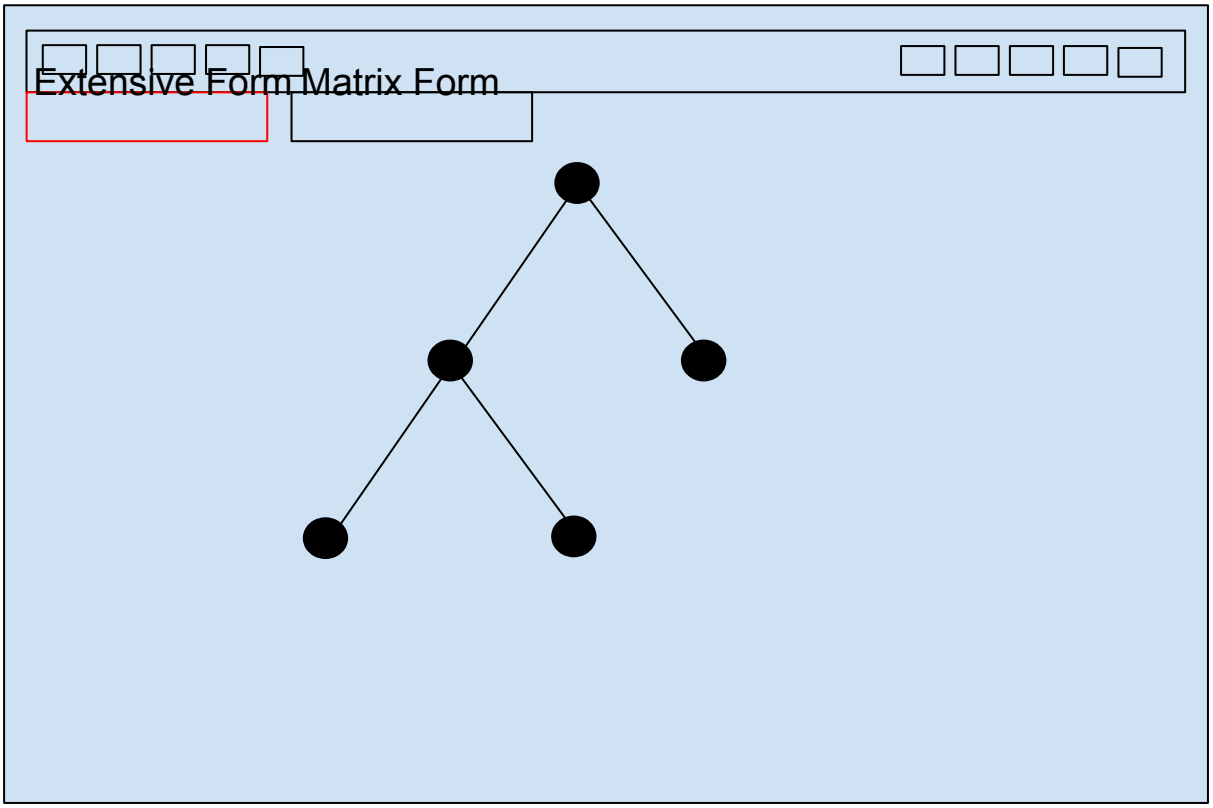
After conversation with my prospective mentors Bernhard von Stengel and Rahul Savani, it is suggested that only the “matrix layout” represents the strategic form as a single panel. If the user wants to input an entire game matrix (e.g. copied from some text), then such a textbox can be popped up by clicking on the player name, for example. This has to be checked carefully because in the extensive game clicking on the player name normally CHANGES the player name; so perhaps a separate button is more appropriate.

Under “flow” the next pages display some of the current data structures and classes as already used in the current prototype JSGTE code.

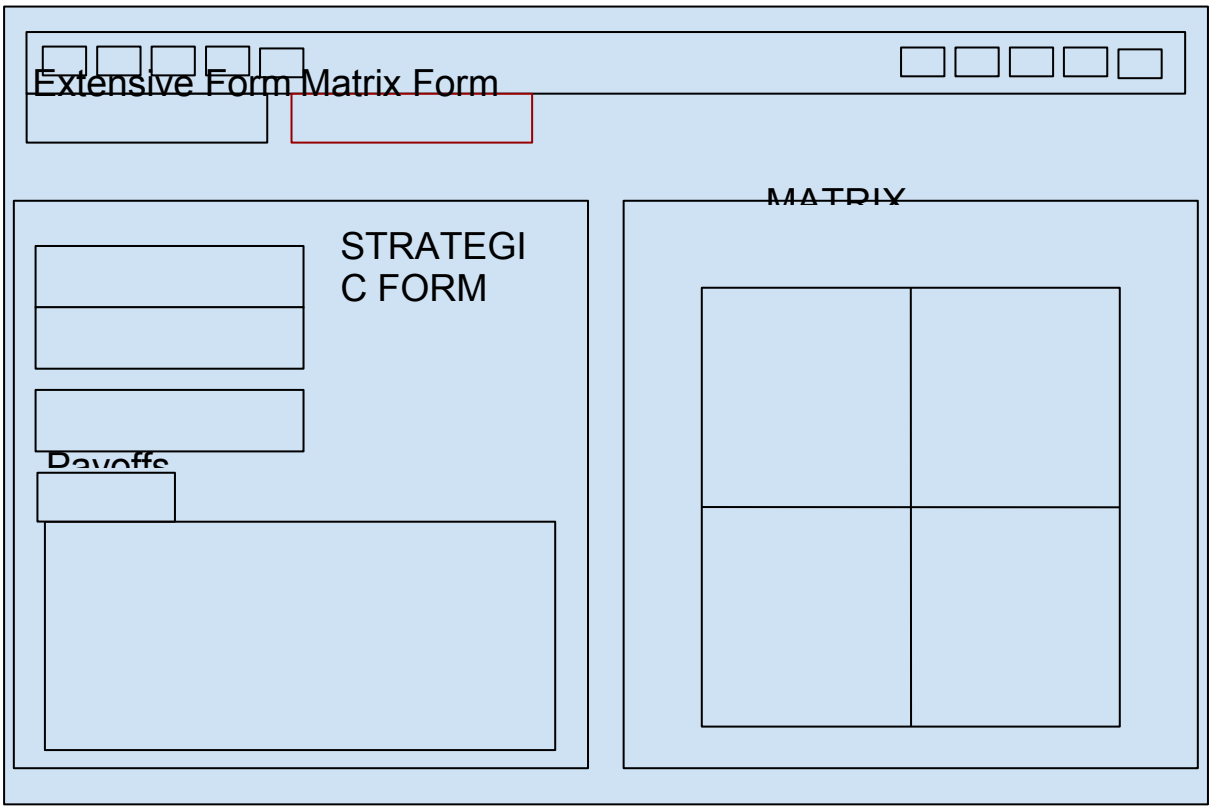
Details of proposed features are presented afterwards.

WIREFRAME

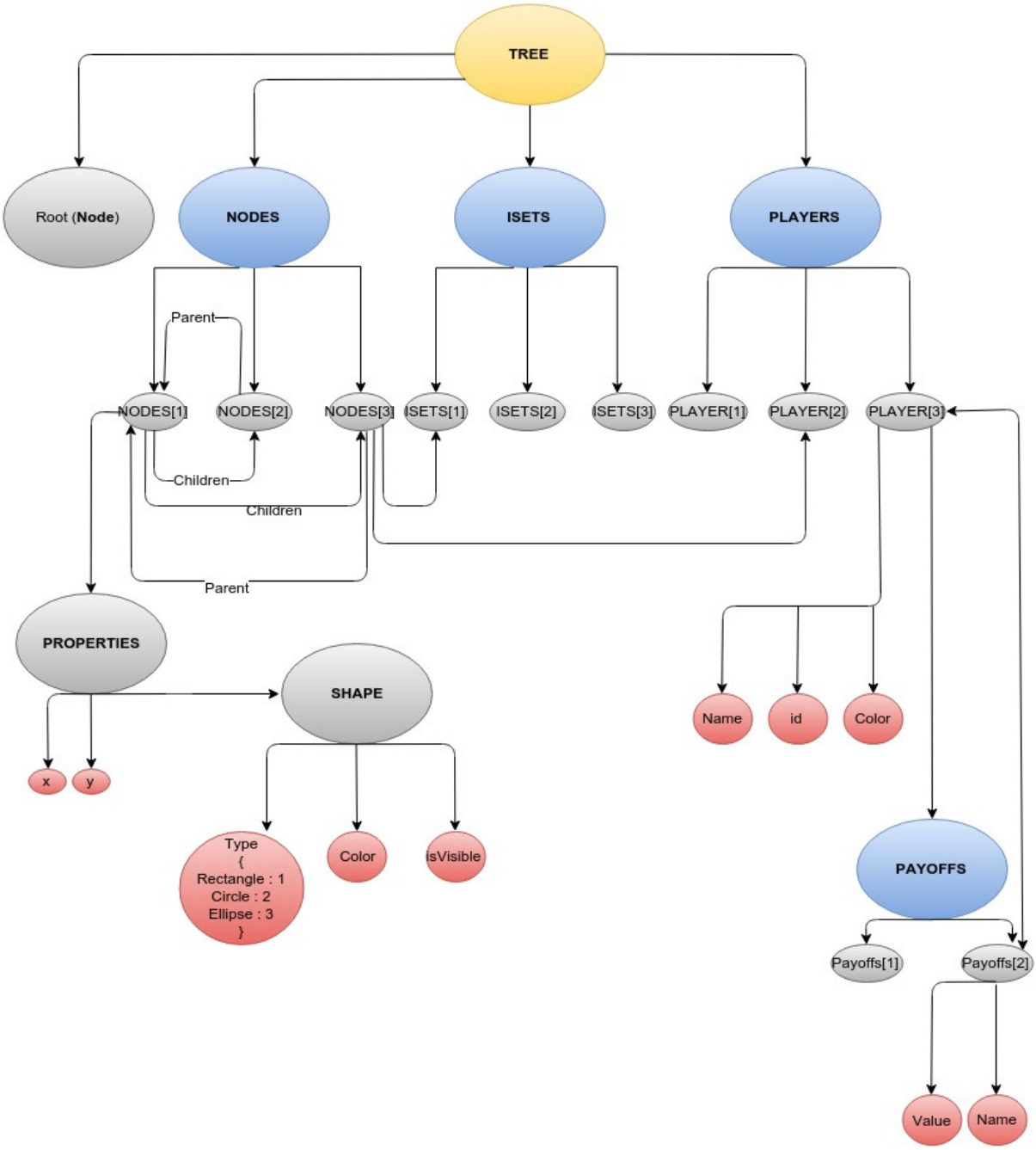
1. Extensive Form



2. Matrix Form



FLOW



FEATURES

1. **Game Tree Input**
2. **Game Matrix Display**
3. **Save to/Load from external XML format**
4. **Export to Graphics Format**
5. **Multiple Actions and Key Control Events**
6. **Reliable Undo/Redo Functionality**
7. **Implement Server Communication**
8. **Add Tests**

Game Tree Input

The first panel would be the Game Tree Input which would allow the user to create the tree on which the game would be played. It would let the user to :

1. Add Nodes
2. Delete Nodes
3. Assign Players to Nodes
4. Merge Nodes
5. Add/Delete Players
6. Undo
7. Redo
8. Settings Panel
    a. Orientation : Let the user change the orientation of the tree to vertical or horizontal. Horizontal Orientations can come in handy in case of unbalanced huge trees. Vertical orientation is present by default.
    b. Assign Color : Let the user assign colors to players.
    c. Modify Shape and properties of Nodes and Lines Connecting the nodes.
    d. Refresh the game.

The tree would be rendered on a canvas and the options to manipulate the tree would be present in a top bar located in the DOM (Check the wireframes).

Game Matrix Display

1. Game Matrix Display Panel will provide an alternate representation of the game to the user.
2. This panel would display the properties and payoffs of the user in tabular and matrix form. The rendering would be done on canvas.
3. The panel would also allow users to directly make and solve matrix games without going through the process of making trees. The matrix input would be taken in a spreadsheet format with key navigations (tab/return/arrow keys).

Save to/Load from external XML format

The tree object is associated with some specific properties and functions. This feature would help export the tree object along with the sub-objects like nodes, payoffs etc. to XML format. It would help store games.
The games could also be loaded later by parsing the XML data.

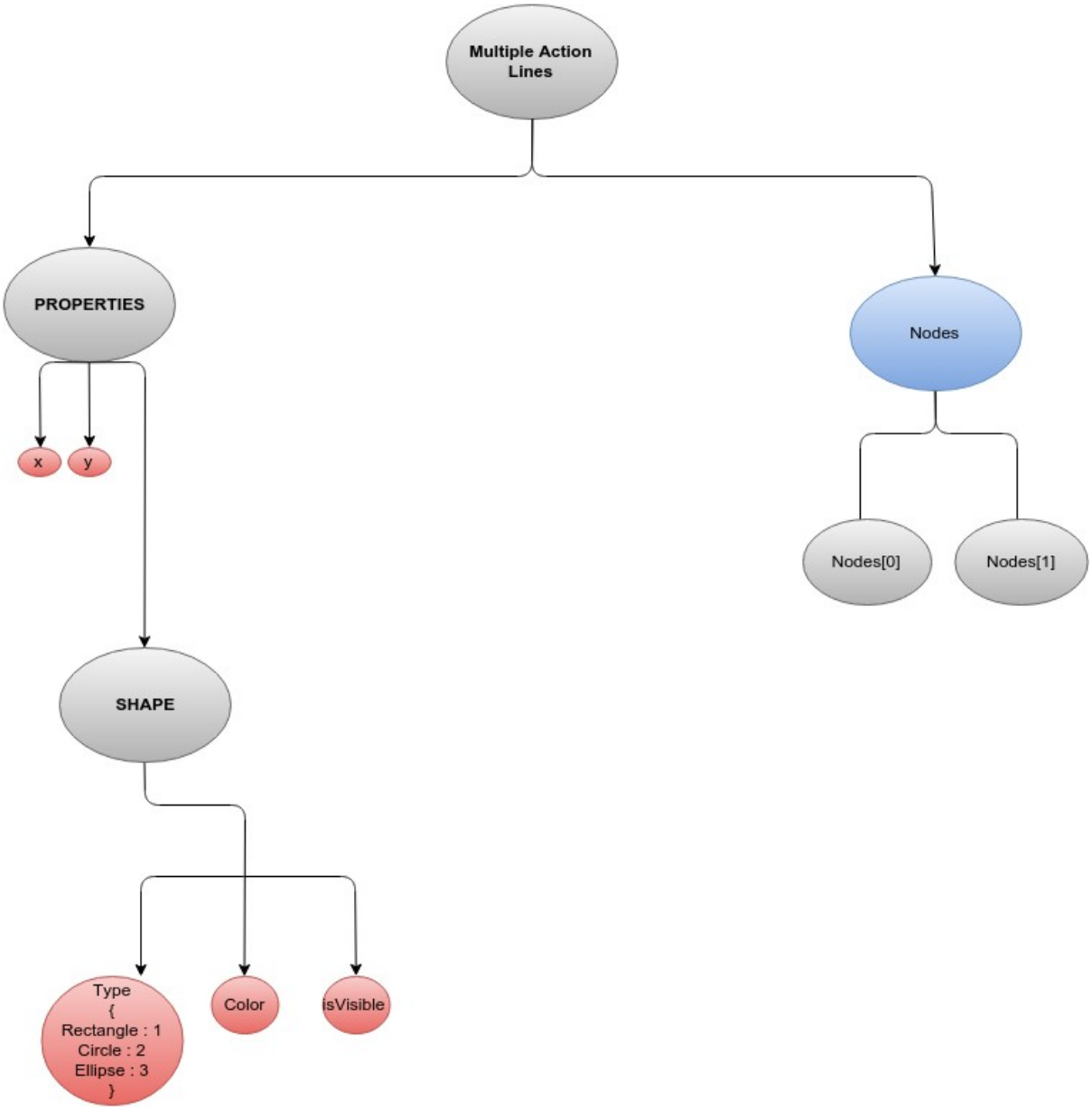
Export to Graphics Format

This feature would render the canvas display to various image formats. The major graphics formats would include png, jpg and svg. In the future, an obj format can also be included to render the tree in 3-D.
Tools to be used : https://github.com/spite/ccapture.js

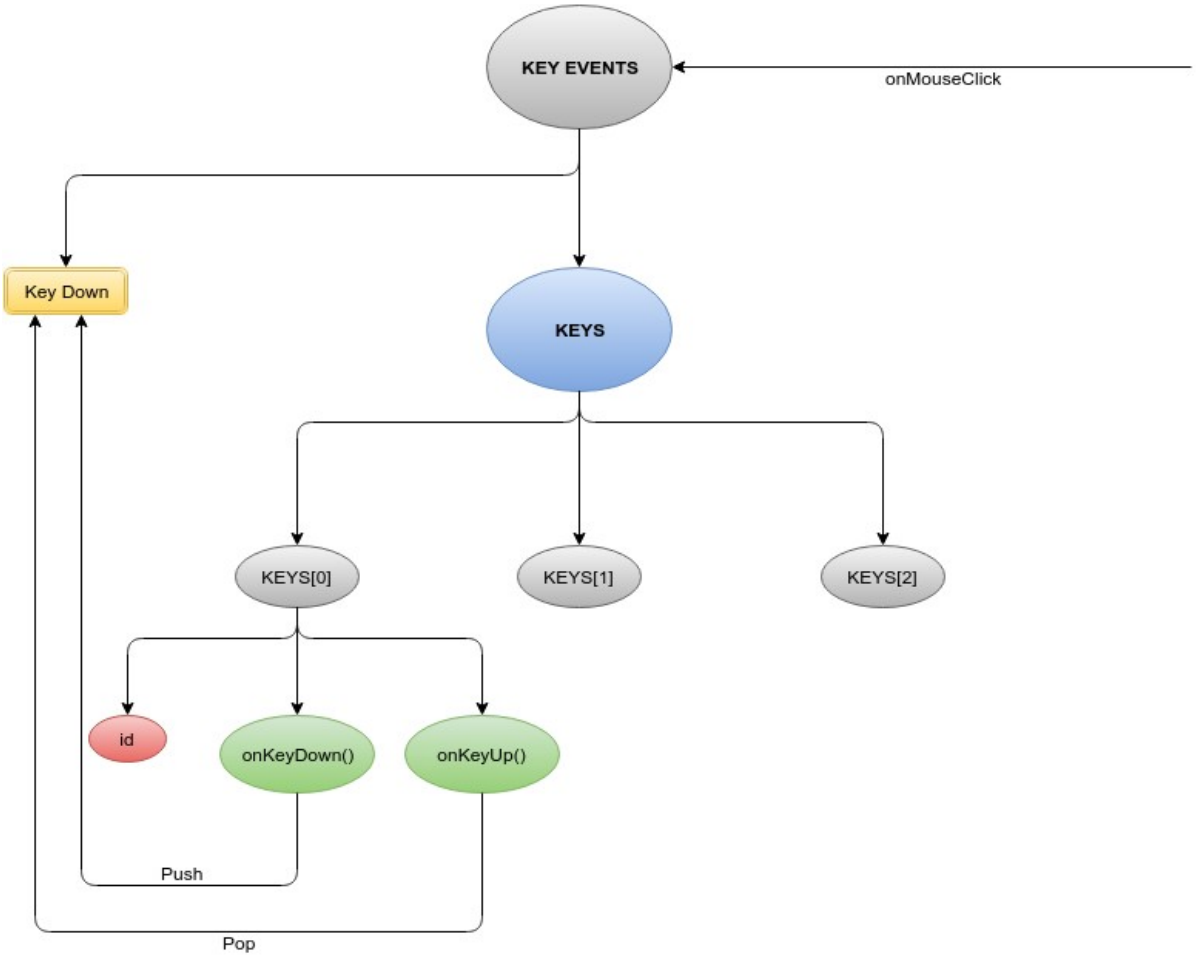
Multiple Action Lines and Key Control Events

This feature would help the user navigate the tree better and make the tree creation process less tedious and fast. The two major steps in this direction include :

1. Multiple Action Lines : In case a user wants to increase the number of children of a particular section of the tree, let's say a particular level of the tree, he can do so in one step rather than going over each node individually. A separate multiActionLines class is included for this purpose.
   It possesses two major properties
   a. Properties : Representing the physical properties of the multiple action lines. In case the nodes are adjacent, then there would be a common shape enclosing the same, else they would be highlighted.
   b. Nodes : An array of nodes contained in the given multiple action lines object.



2. Key Control Events : The key control events would help the user navigate the tree faster and without the mouse. It would also let the user select multiple nodes and change the properties of these nodes together.
   Use Case :
   a. Ctrl + Z : Undo
   b. Ctrl + Y : Redo
   c. Ctrl + Left Mouse Click : Select Nodes
   d. Shift + Drag : Select Multiple Nodes in the Given Area
   e. Navigation :
      - Up : Move to the Node above
      - Down : Move to the left most Node below
      - Left : Move to the left Node at the given level
      - Right : Move to the right Node at the given level



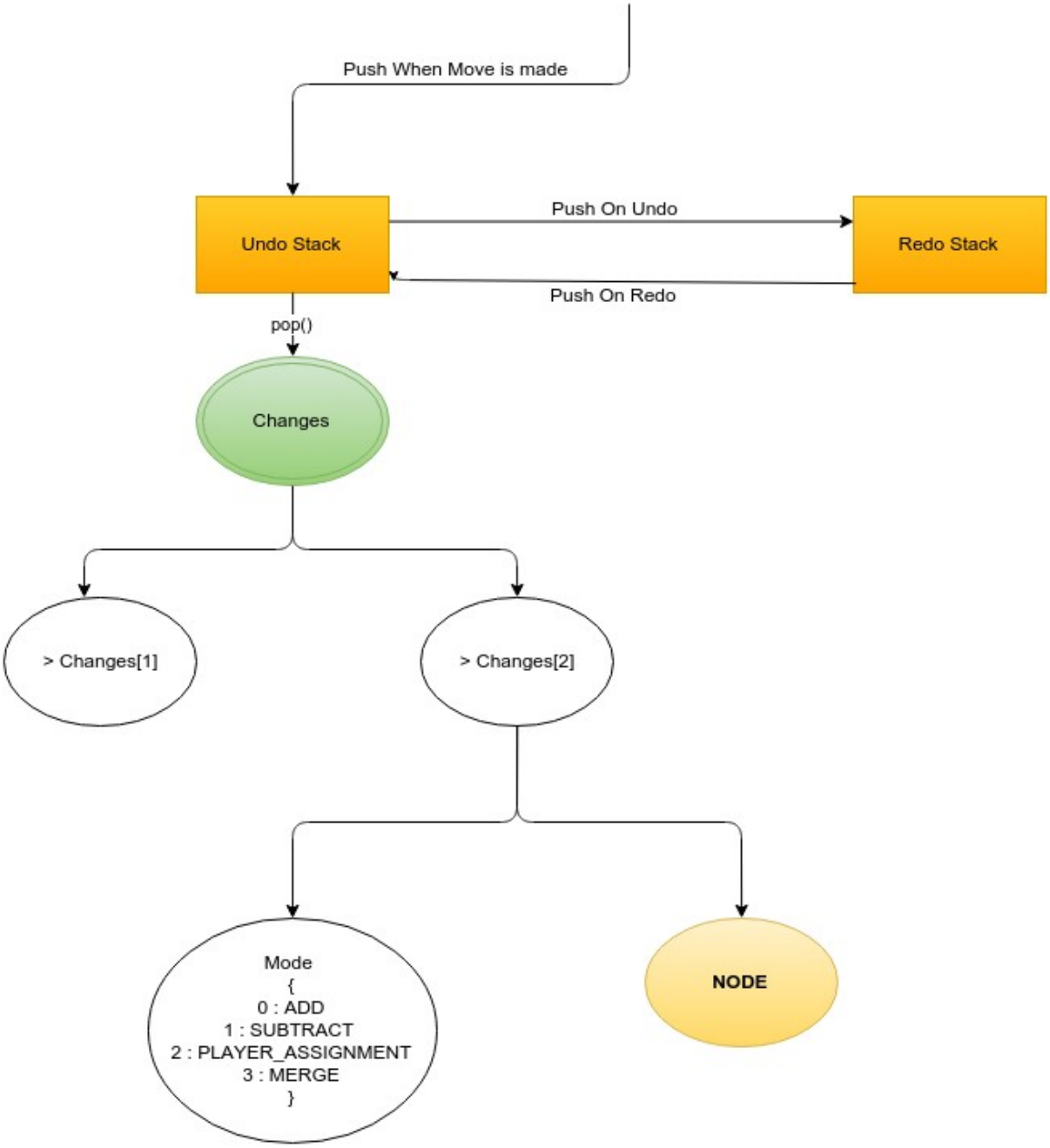
Reliable Undo/Redo Functionality

Allow the user to undo/redo a move while manipulating the tree.

Implementation

Create a global stack that stores the changes that are made in a particular move. Whenever a user wants to undo a move, pop the stack and based on the changes made in that move, revert to the previous state.

1. Undo Stack : Whenever a che is made to the structure of the tree or in terms of the player assignment, a Changes array would be pushed into the Undo Stack.
2. Redo Stack : If the user chooses to undo a particular move, the change corresponding to this revert would be pushed into the Redo Stack.
3. Changes : An array consisting of Change objects which have been changed during that particular move.
4. Change : The Change Class consists of two main parameters
   A. Node : A reference to the node that was changed in this particular move.
   B. Mode : Representing the change that the current node went through during the move



Implement Server Communication

Since the computation of equilibrium for larger games is computationally intensive, hence the computation of the nash equilibria will take place on a public server. This feature would involve implementing communication among the server and the browser for solving the given game. Would require data received from the server to be parsed and represented accordingly.

Add Tests

Jsgte would be implemented as a very mature and modular project and to make development easy in the future, Tests are necessary. This would be the last task to be taken up during the project.

Unit Testing using Chai :

I'll be using Chai expectation library to make assertions against the javascript code. Testing would be done in a node environment. Since the number of classes and their dependencies among them are enormous, Assertion testing would help make sure the quality of the code is maintained in the future.

Implementation, Progress and Tentative Schedule

1. **Game Tree Input**
   Work on this feature has been almost completed. A few changes were suggested by the mentors to make the user experience better, like better representation of hover events and click events which need to be implemented. Already discussed with the mentors.
   Timeline : mid May to June first week.
2. **Game Matrix Display**
   This feature is to be implemented from scratch.
   Timeline : mid May to June first week.
3. **Save to/Load From external XML format**
   To be implemented from scratch.
   Timeline : June first week - June third week.
4. **Export to graphics format**
   Already implemented in the flash version but not present in jsgte. A lot of open source tools are available to implement this feature.
   Timeline : June first week to June third week.
5. **Multiple actions and key control events**
   Already partly implemented. Multiple actions (such as adding a node or creating information sets) for all nodes on a level when the mouse pointer hovers at that level, for higher input speed.I already improved on a bug of this behaviour in the current implementation. This should also be amended for other operations, such as dissolving information sets. I am also in conversation with my mentors about these features.
   Timeline : Mid June to July first week.
6. **Reliable Undo/Redo functionality**
   I submitted a prototype presenting the approach that would be followed while implementing the undo functionality. It was reviewed by some of the mentors and it is a general approach that is followed while implementing undo functionality. The PR can be found here : https://github.com/gambitproject/jsgte/pull/66
   Timeline : July first week to July third week.
7. **Implement Server Communication**
   Right now the project has no interface whatsoever to communicate with the public

server that will solve the Nash equilibria of the game. To be built from scratch using ajax.
Timeline : July third week to August second week

8. **Add Tests**
   To be added in the last week along with final bug fixes and code clean up.

By mid term evaluation, I aim to finish the first four features completely and be on the verge of finishing the fifth one. This would be a good kickstart into the program.

Personal and Contact Details

Name
Harkirat Singh
Email
harkirat96@gmail.com
Github
https://github.com/hkirat
Location
Roorkee, Uttarakhand, India
Working Hours
11:30-13:30, 15:00-19:00, 22:00-05:00 (IST) 6:00-8:00, 9:30-13:30, 16:30-23:30 (UTC)

Skype Name :
harkirat961

Appear.in :
https://appear.in/hsk

Background

I love solving algorithmic puzzles and have interest in mathematics and game theory.
I have developed software in many programming languages but my primary interest lies in javascript.
Most of the projects that I took up are not open source but I'll be more than happy to share the code with the mentors.
Some of the projects that I've worked on :

1. Chess Wars : A chess playing platform live on the intranet of IIT Roorkee where people can play chess against their friends and against a bot. Front end is mainly javascript and CSS and the backend is written in Node. It involves the use of Socket.io to handle asynchronous connections. The bot is written in python.
2. Hubble : A link sharing platform for the people of IIT Roorkee. Live on the internet. Backend is written in PHP. https://hubble.sdslabs.co/ .

I am the Joint Secretary of a campus group called SDSLabs (https://sdsabs.co) where we develop software round the year for the people of IITR.
I am pursuing B.Tech in Computer Science and Engineering from Indian Institute of Technology, Roorkee. I am currently in my sophomore year.
I have good experience in version control systems and have previously contributed to open source. **In particular, I have already contributed some patches to the current JSGTE code.**
Links :

1. https://github.com/hkirat
2. https://github.com/gambitproject/jsgte/pull/66
3. https://github.com/gambitproject/jsgte/pull/71
4. https://github.com/processing/p5.js/commits/master?author=hkirat

Motivation

I love solving algorithmic puzzles. Also, I love building softwares and websites, primarily in Javascript. I've worked with Node.js and Javascript on some mature projects and have explored javascript to the core. Hence, I find this project exciting and interesting. Deciding and working on the structure of this project would be something pretty interesting.

Need

To provide easy access to the functionalities of Gambit by providing a browser interface. Gambit needs to be installed and since people tend to skip over the tedious job of downloading, installing and setting up a software, the reach of gambit is restricted.
With the power of browser languages like HTML5 and Javascript, the same functionalities can be provided in a browser providing easy access and better reach to the gambit resources.This project would help open new possibilities for gambit and provide much greater outreach to it than it already has.

Summer Plans

I have no commitments in the summer. I'll be staying back home for the most part of it. I have mentioned my typical working hours above and on an average will be able to spend 40 hours per week on the project.